PREPSTACK

Strategic Differentiation & Product Improvement Blueprint

How to build a distinct, evidence-driven job-preparation platform

A detailed strategic review of the current improvement plan, the competitive positioning against TUF, and a practical path to make PrepStack meaningfully different without copying competitors.

Prepared: 8 November 2026

Working document • Strategy, product design, roadmap and validation

1. Executive summary

Your goal should not be to build a smaller version of TUF, ZENKAI, Planly, or another preparation platform. PrepStack should solve the same broad student problem from a different product angle: help a learner become demonstrably ready for a specific role, then help them communicate that readiness to recruiters and interviewers.

The strongest direction is to evolve PrepStack into an **evidence-driven job-readiness system**. A user chooses a target role or supplies a job description. PrepStack diagnoses the skills involved, turns the gaps into a practical plan, connects each task to relevant learning and practice, and records evidence that the user can actually perform the skill. The system then produces an explainable readiness view—not a mysterious AI-generated percentage.

Your current IMPROVEMENT_PLAN.md is a strong quality and integration roadmap. It addresses thin content, disconnected resources, weak progress tracking, unsaved notes, and the lack of a unified plan. Those are worthwhile improvements. However, most of them are expected features in a modern preparation product. They improve completeness, but they do not automatically create differentiation.

The strategic change is not to add dozens of new features. It is to connect the features you already planned through one shared skill model and one evidence loop.

> **Product thesis:** PrepStack helps a learner move from “I want this job” to “Here is the evidence that I am ready for this job”—with a clear, achievable path between those two states.

The central product loop

1. Target: choose a role, placement goal, or paste a job description.

2. Diagnose: identify the competencies required and compare them with the learner’s current evidence.

3. Prioritize: identify the most important gaps and prerequisites.

4. Execute: create a realistic plan using PrepStack’s DSA, CS, roadmap, project, resume, and behavioral resources.

5. Verify: collect evidence through practice outcomes, quizzes, checkpoints, project artifacts, and story examples.

6. Communicate: turn verified work into stronger resume bullets and interview stories.

7. Reassess: update readiness as skills improve, become stale, or remain unproven.

Role Analyzer becomes the diagnostic entry point. My Plan becomes the execution layer. The shared competency and evidence model becomes the differentiating engine that connects them.

2. Competitive framing: what TUF already offers

A fair comparison matters. TUF’s public positioning is broader than a DSA tracker. Its current public pages describe company-specific preparation, DSA and SQL practice, editorials, learning content, Planly weekly sprints, progress tracking, time tracking, and AI assistance through TUFY. Public descriptions also mention CS subjects and other placement-preparation areas. Therefore, it would be inaccurate to position PrepStack as different simply because it has a planner, AI assistance, company questions, or a wider collection of subjects.

The public material establishes what TUF advertises; it does not establish every detail of its internal implementation or prove that a capability is absent. Treat the points below as strategic observations and hypotheses to investigate—not confirmed product defects.

What this means for PrepStack

• **Do not compete on feature-count alone.** A larger list of sheets, questions, notes, or planner controls is easy to imitate and difficult to maintain.

• **Do not claim a planner is unique.** TUF publicly promotes Planly and weekly sprint workflows.

• **Do not claim AI is unique.** TUF publicly promotes AI assistance. Your advantage should come from the way PrepStack connects information and evidence, not merely from adding a chatbot.

• **Do not claim company preparation is unique.** TUF publicly promotes company-specific questions and patterns.

• **Compete on the quality of the job-readiness workflow.** Show how every diagnosed gap becomes a concrete action and how completion is supported by evidence.

Potential weaknesses to investigate—not assume

Area	Hypothesis to test	How PrepStack could respond
Breadth and navigation	A broad platform can create choice overload for some learners. This is a hypothesis, not a proven TUF defect.	Give each learner a role-specific route through the content instead of exposing every resource as an equally important choice.
Plan completion vs mastery	Public Planly descriptions emphasize planning, tasks and progress. Public pages do not clearly establish how deeply completion is verified as skill mastery.	Separate task completion from demonstrated competency. Attach a quiz, practice result, checkpoint or artifact to important skills.
Proof of readiness	The public positioning does not make a cross-resource, evidence-backed job-readiness record its central promise. This does not prove the product lacks internal evidence features.	Make explainable evidence per role competency the centre of PrepStack's experience.
Portfolio connection	It is not clear from public marketing whether learning progress is systematically connected to a learner's own project artifacts and resume claims.	Connect projects, GitHub/demo links, acceptance criteria and resume bullets to the competencies they demonstrate.
Content choice	A large catalogue may still leave a beginner unsure what to do next. Validate this with actual students before treating it as a weakness.	Use diagnostic results and prerequisites to recommend the next smallest useful action.

Important: a competitor audit should distinguish three evidence levels: (1) directly observed in the product, (2) stated on an official public page, and (3) inferred or hypothesized. Never turn level 3 into a factual claim.

3. What is already strong in your improvement plan

A. You have identified real product friction

• DSA progress is currently too shallow: solved links alone do not provide dates, streaks, per-sheet visibility or useful weekly feedback.

• Problem notes stored only in localStorage are fragile across devices and account changes.

• CS notes have uneven depth, with DBMS, CN and OOP needing more substance and OS already relatively deeper.

• Roadmap steps are currently too thin to guide execution: a title, one line and external links are not a complete learning unit.

• Resume and behavioral guides need examples, practice structure and reusable outputs.

• Role Analyzer has a useful 120-skill catalogue, but a portion of its recommendations lead to thin or missing PrepStack learning assets.

• Dashboard reporting needs to move from aggregate totals toward useful per-sheet and daily context.

• 469 server tests are a meaningful foundation, while client tests and CI remain gaps.

B. Your constraints are healthy

• No new AI cost for core features.

• No unnecessary video platform, custom compiler, judge, coins, store, ads or paid tiers.

• Original content rather than copied competitor material.

• Accuracy and review over inflated content volume.

• One source of truth for counts and claims.

• A staged release with student validation and a fallback point.

These constraints are not limitations to apologize for. They are product decisions that help keep the project focused, affordable and maintainable.

4. The differentiation: an Evidence-Driven Job Readiness System

4.1 The problem it solves

Many learners can say they watched a course, read a topic, checked a problem as solved, or completed a roadmap step. Those actions do not always prove they can explain the concept, solve a new problem, build something with it, or discuss it in an interview. PrepStack can address the gap between **activity** and **readiness**.

4.2 The shared competency graph

Create one canonical competency catalogue that connects all major parts of PrepStack. A competency is a skill or capability that can be required by a target role and supported by one or more learning or evidence items.

Object	Example	Purpose
Role	Backend Developer Intern	Defines the target outcome.
Competency	Database indexing	A skill required by one or more roles.
Learning asset	DBMS note: indexes and query plans	Explains the concept.
Practice asset	SQL exercise or interview question	Lets the learner apply it.
Checkpoint	Explain when an index may not help	Checks understanding beyond reading.
Evidence	Quiz result, practice status, project link, explanation	Supports a readiness judgement.
Resume/story link	Project bullet or STAR story using database optimization	Helps communicate the competency.

Use stable competency IDs rather than matching resources by their visible titles. For example, `dbms.indexing` can be referenced by a note, roadmap step, quiz, project rubric, Role Analyzer recommendation and readiness report. If a title changes, the relationship remains intact.

4.3 What counts as evidence?

• **Knowledge evidence:** a reviewed quiz or short explanation demonstrating conceptual understanding.

• **Problem-solving evidence:** a dated attempt and outcome, with optional difficulty, confidence and revisit status. Since PrepStack links to external practice sites, do not claim automatic code verification unless you actually integrate a reliable source.

• **Practical evidence:** a project checkpoint, deployed demo, repository link, README, screenshot or documented implementation decision.

• **Communication evidence:** a resume bullet, project explanation or STAR story mapped to the competency.

• **Recency evidence:** when the learner last demonstrated the skill. Older evidence may need revision rather than being treated as permanently current.

Do not over-engineer a readiness score at the beginning. Start with transparent states such as **Not assessed**, **Learning**, **Practised**, **Demonstrated** and **Needs refresh**. Explain why each state appears.

4.4 Avoid false precision

A single percentage such as “87% job-ready” can look scientific without being meaningful. If you later introduce a score, show its components, evidence coverage, missing evidence and confidence. A learner should be able to answer: “Why does PrepStack think I am strong or weak in this skill?”

> **Recommended first version:** a competency table with status, last evidence date, linked evidence and next recommended action. Add scoring only after you have enough real usage data to validate that the score is useful.

5. How to adapt your current plan without exploding scope

You do not need to discard your 9–10 week plan or add a separate large product. Add a thin shared data model and a few carefully chosen behaviours to the work you already intend to do.

5.1 Track A — Content improvements

A1. CS notes: make them competency-linked

• Keep your planned structure: overview, topic explanations, interview questions and answers, common mistakes, MCQs with explanations, internal links and reading time.

• Add a stable competency ID to each topic and question.

• For each important topic, include at least one application checkpoint: explain it, compare two approaches, diagnose a failure case, or solve a small scenario.

• Use the same competency IDs in Role Analyzer and roadmaps. This prevents a skill from being represented differently in every feature.

• Keep the planned order: DBMS, CN, OOP, deepen OS, then System Design basics. Do not chase word-count targets at the expense of correctness.

A2. Roadmaps: turn steps into outcomes

Your planned roadmap object is a good foundation. Add an explicit outcome and evidence expectation to each meaningful step.

• **Goal:** what the learner should be able to do after the step.

• **Topics:** a small, ordered checklist tied to competency IDs.

• **Resources:** two or three reviewed free resources, not a giant link dump.

• **Practice:** a task that requires application.

• **Checkpoint:** a simple way to show understanding.

• **Project connection:** a relevant mini-project or project-generator brief where appropriate.

• **Time estimate:** an honest range with assumptions, not a guarantee.

A roadmap step should not be marked “mastered” merely because the learner opened its links. Distinguish reading, practice and demonstrated outcome.

A3. Resume guide: connect claims to proof

• Keep your weak/strong examples, role variants, recruiter scan notes and one-page guidance.

• Add a proof question to each bullet: what did the person build, improve, measure or decide?

• Show how a project artifact supports the bullet: repository, deployed demo, feature, test or measurable result.

• Make it clear that fictional examples are illustrative and must not be copied as personal achievements.

• Link the guide to Role Analyzer, but avoid generating inflated claims that the user cannot defend.

A4. Behavioral guide: build a reusable story bank

• Expand the question categories as planned: conflict, failure, learning, ownership, deadlines, ambiguity, feedback, leadership and motivation.

• For each question, show interviewer intent, a STAR structure, a fresher-friendly sample and common mistakes.

• Let the learner save five or six real stories and map each story to multiple competencies and question types.

• Capture context, the learner's specific action, result, learning and likely follow-up questions.

• Do not encourage invented metrics or fictional experiences. Help users describe real contributions clearly.

A5. Role Analyzer: close the content gaps

The 120-skill catalogue is a valuable integration point. Map every skill to the best available internal asset, but do not force a weak link just to reach 100% coverage. Use explicit states such as "lesson available", "external resource", "practice available" and "content planned".

• Use one canonical competency ID for every skill.

• For each skill, link the most useful note, roadmap step, practice checkpoint and project evidence option.

• Show the learner the reason for the recommendation and the expected effort.

• Make the Analyzer diagnostic: it identifies gaps. Do not let it silently become a second planner.

• Add an "Add to My Plan" action that passes the selected competency and source metadata into the plan.

5.2 Track B — Feature improvements

B1. DSA progress: improve the meaning of progress

Your plan to store `solvedAt` and migrate legacy records is a sensible first step. Consider a lightweight status model that does not require building a coding judge:

Field	Why it matters	Keep it simple
status	Distinguishes activity from outcome.	Not started / attempted / solved / revisit.
solvedAt or attemptedAt	Enables honest time-based views.	Store timestamps; treat migrated unknown dates as unknown.
confidence	Signals whether a result felt repeatable.	Optional low/medium/high self-rating.
difficulty	Adds context to the accomplishment.	Use the source's difficulty where reliable; otherwise omit.
competencyIds	Connects problems to the skill graph.	Use a small curated mapping, not automatic guesses.

Keep the IST day-boundary rule explicit and test daylight/time-zone edge cases where relevant. A streak is a habit indicator, not proof of mastery. Avoid punishing learners for old records that lack dates.

B2. Saved notes

• Sync notes to the account while retaining a local offline copy if that is part of your intended experience.

• Make the one-time browser-note import transparent and idempotent.

• Provide search, deletion and clear privacy language.

• Do not silently overwrite a newer server version with stale local data.

• Treat notes as private user content and include them in account deletion.

B3–B4. Dashboard and reusable checklist

Your shared checklist abstraction is a good architectural move. Keep task state separate from evidence state. For example, a user may complete “read DBMS indexing” but still need a quiz or practical checkpoint before the competency is considered demonstrated.

• Preserve the distinction between completed task and verified competency.

• Store task source metadata so the user can see whether it came from Role Analyzer, My Plan, a roadmap or a manual addition.

• Use one task model for the report and My Plan rather than maintaining duplicate completion logic.

• Make Today useful: show a small number of prioritized actions, time estimates and a clear way to defer or reschedule.

B5. My Plan: make it adaptive, not just scheduled

The planned single active plan is sensible. The key is to avoid building a generic calendar or checklist clone. The plan should be generated from competency gaps and should respond to demonstrated performance.

• Accept the inputs you already identified: target role/JD, deadline, available time and starting context.

• Use deterministic rules to prioritize prerequisites, high-impact gaps and overdue evidence. No new AI call is required.

• Estimate tasks with transparent 20/35/60-minute buckets, but allow the learner to adjust them.

• If a learner struggles with a checkpoint, add a prerequisite or revision task. If the learner demonstrates the competency, reduce or skip redundant tasks.

• If a day is missed, move work forward without creating an impossible backlog. Preserve the learner’s ability to remove, move or rebuild tasks.

• Keep one active plan and make the consequences of replace/merge explicit before confirmation.

• Keep task provenance: source competency, source asset, estimated time and reason for inclusion.

> Launch only after the prerequisites you listed—stronger notes, roadmaps, progress and shared checklist—are ready. Your proposed student and job-switcher validation gate is important. If testing shows that the plan is mostly a chore list, stop at B4 and improve the underlying content/evidence connections first.

B6–B9. Quality, quizzes, profiles and tests

• Use consistent empty, locked, loading and limit states. These small details strongly affect perceived reliability.

• Build quizzes from reviewed content; keep answer explanations and avoid ambiguous questions.

• Keep public profiles off by default if you add them later. Make visibility and data exposure explicit.

• Add client tests and basic CI. Protect the core user flows: analyzer mapping, plan generation, checklist state, migration, note sync and deletion.

• Automate checks for broken internal links, missing competency mappings, duplicate IDs and stale README counts.

6. Make the existing AI project generator more valuable

You already have AI project generation. Rather than adding another AI assistant or another paid inference path, improve the usefulness of the output and connect it to role competencies.

A stronger project brief should contain

• The target role and competencies the project is intended to demonstrate.

• A real user problem and a specific target user—not only a technology stack.

• Core scope and explicitly excluded scope.

• Functional requirements and acceptance criteria.

• Architecture outline, data model and API boundaries where appropriate.

• Milestones with testable checkpoints.

• Security, reliability and performance considerations appropriate to the project.

• Deployment expectations, README outline and demo checklist.

• A rubric explaining what a reviewer should be able to verify.

• Resume-bullet prompts that require real measured outcomes rather than fabricated impact.

The AI can help draft the initial brief, while reusable templates, competency mapping, acceptance criteria and progress tracking can be deterministic. This improves product value without requiring a new AI feature or a new AI cost for every planner action.

7. Product architecture: a practical starting model

Do not build a complex graph database just because the product concept uses the word “graph”. A relational model with stable IDs and join tables is likely enough for the first version.

Concept	Possible records / fields	Notes
Competency	id, title, category, description, level, prerequisites	Canonical source of truth for skill identity.
Role mapping	roleId, competencyId, importance, rationale	Maps a role or JD to required competencies.
Asset mapping	competencyId, assetType, assetId, relationship	Connects notes, roadmap steps, questions and projects.
User evidence	userId, competencyId, evidenceType, status, evidenceRef, createdAt	Stores what the learner has demonstrated or self-reported.
Plan task	planId, competencyId, source, dueDate, estimate, status	Represents an action, not proof of mastery.
Checkpoint result	userId, checkpointId, result, attemptAt, explanation	Supports evidence and adaptive planning.

Important data rules

• Use stable IDs and validate references at the API boundary.

• Keep task completion, evidence and competency status as separate concepts.

• Record provenance: where an item came from and why it was recommended.

• Avoid storing sensitive information from job descriptions longer than necessary; explain retention and deletion.

• Make migrations reversible where practical and test old data carefully.

• Do not derive a mastery claim from a user merely opening a page or clicking a checkbox.

8. What to measure: product outcomes, not vanity metrics

Track whether PrepStack helps learners make progress toward a target role. Page views, number of notes and total checkmarks are not enough.

Metric	What it tells you	Caution
Time to first useful plan	How quickly a new user reaches a relevant next action.	A fast plan is not useful if it is generic.
Gap-to-action conversion	Whether diagnosed gaps lead to meaningful tasks.	Do not count a click as successful learning.
Checkpoint completion	Whether learners attempt the practice tied to learning assets.	Completion does not always equal mastery.
Evidence coverage	How many important role competencies have supporting evidence.	Coverage should be weighted by role importance.
Plan usefulness rating	Whether recommended tasks feel relevant and achievable.	Collect short qualitative feedback too.
Return to revise / refresh	Whether learners revisit weak or stale competencies.	Avoid optimizing for addictive streak behaviour.
Resume/interview defensibility	Whether users can explain claims using actual work.	Validate with mock interviews or external reviewers.

9. Validation plan before expanding scope

Do not rely only on your own experience as the product builder. Test the workflow with a small group before committing to the full My Plan implementation.

Suggested test

1. Recruit 3–5 students preparing for placements and 2–3 job switchers, as your plan already proposes.

2. Ask each person to choose one real target role or provide a real job description.

3. Observe whether they understand the skill-gap report without your explanation.

4. Ask them to choose the next task. Record where they hesitate, what they ignore and whether the task feels relevant.

5. Have them complete one learning asset and its checkpoint. Ask whether the checkpoint genuinely tests the skill.

6. Ask whether the resulting evidence would help them explain their ability in an interview.

7. After a few days, ask whether the plan remained realistic and what they changed.

Questions to ask

• Did the Analyzer identify gaps that felt relevant to the target role?

• Could the user tell why each recommendation appeared?

• Was the next action clear without a walkthrough?

• Did the task estimate fit the user’s available time?

• Did the checkpoint test understanding or merely recall?

• Would the user trust the readiness status? What evidence would they want to see?

• What did the user do outside PrepStack that the product should acknowledge?

Do not ask only “Do you like it?” Observe actual behaviour. A learner saying a feature is useful is weaker evidence than watching them use it successfully.

10. Recommended implementation sequence

This sequence preserves your existing scope and places the differentiating foundation before the planner becomes large.

Stage	Focus	Deliverable / exit condition
0. Product foundation	Define product thesis and competency IDs.	A small canonical taxonomy and mapping rules agreed before content expansion.
1. Content quality	DBMS, CN, OOP, OS; roadmaps; resume and behavioral content.	Accurate, reviewed learning assets with stable competency links.
2. Progress foundation	DSA dates/status, notes sync, dashboard, shared checklist.	Reliable user progress with migration, privacy and tests.
3. Evidence layer	Checkpoint results, project artifacts, story mapping, competency status.	A learner can see what is learned, practised and demonstrated.
4. My Plan pilot	Deterministic task generation and light adaptation.	Small user test confirms tasks are relevant, achievable and connected to evidence.
5. Readiness report	Explainable competency view for one target role.	Every status has a reason, evidence reference and next action.

If the schedule becomes tight, protect the quality of the content, data model, migration, privacy and tests. Defer public profiles, elaborate scoring, complex analytics and nonessential UI polish.

11. What not to build right now

• **Do not build a Planly clone.** Your plan should be a role-specific execution layer, not a standalone generic planner.

• **Do not add a generic AI chatbot.** It adds cost and does not solve the core differentiation problem.

• **Do not build a custom code judge.** It is expensive to maintain and outside your stated non-goals.

• **Do not add coins, leaderboards or streak pressure.** They can increase activity without improving readiness.

• **Do not add a community feed yet.** It introduces moderation, safety and cold-start challenges.

• **Do not expand to more role tracks prematurely.** Improve the accuracy and completeness of the existing six roadmaps first.

• **Do not publish a readiness percentage without evidence.** Explainability and trust matter more than impressive-looking numbers.

• **Do not write content just to hit word-count targets.** A concise, correct explanation is better than padded notes.

12. Risks and safeguards

Risk	Why it matters	Safeguard
Competency taxonomy becomes too large	Mapping and maintenance become a project of their own.	Start with the 120 skills already used by Role Analyzer; normalize and deduplicate before adding more.
False mastery signals	Users may trust a misleading readiness label.	Separate self-report, practice outcome and reviewed checkpoint evidence.

Risk	Why it matters	Safeguard
Content accuracy drift	Incorrect technical guidance damages trust.	Source claims, test code examples and use independent review for important topics.
Planner scope creep	A complex planner can delay core improvements.	Launch only after prerequisites; pilot deterministic planning first.
Privacy and account deletion gaps	Saved notes, stories and evidence may contain personal data.	Document retention, deletion, visibility and export behaviour before launch.
Competitor assumptions	An inaccurate comparison leads to weak strategy.	Label observed facts, public claims and hypotheses separately; retest periodically.

13. Your product positioning

A positioning statement should explain the outcome and the method—not list every feature.

> **Suggested positioning:** “PrepStack is a role-focused job-preparation workspace that turns your target job into a clear skill-gap plan, connects each gap to practical learning, and helps you build evidence you can use in your projects, resume and interviews.”

Messaging pillars

• **Know what matters:** understand the competencies required for a target role.

• **Know what to do next:** get a realistic, prioritized plan based on your gaps and available time.

• **Know what you can prove:** connect practice, checkpoints and project work to the skills you claim.

• **Explain your work:** convert real evidence into clearer resume bullets and interview stories.

This positioning does not say that PrepStack has more content than competitors. It says PrepStack connects preparation to demonstrable readiness.

14. Final recommendation

Keep the majority of your current improvement plan. It correctly addresses the product’s weakest foundations. Add a thin but deliberate differentiation layer: shared competency IDs, evidence records, and a readiness view that distinguishes activity from demonstrated ability.

The most important strategic decision is this: **PrepStack should not measure only how much a learner has completed. It should help them understand what they can demonstrate for the role they want.**

That means your next steps should be:

1. Write the product thesis at the top of IMPROVEMENT_PLAN.md.

2. Define a minimal competency schema using the existing Role Analyzer skill catalogue.

3. Map A1/A2 content to those IDs as you improve the notes and roadmaps.

4. Keep task completion separate from evidence and mastery.

5. Add one small evidence checkpoint to the most important learning assets.

6. Build the shared checklist and My Plan on top of that foundation.

7. Pilot the flow with real learners before adding advanced scoring or more features.

Your plan already has discipline: clear non-goals, migration considerations, tests, content review and a staged launch. The opportunity is to make every planned improvement serve one recognizable product idea. That is how PrepStack becomes distinct without copying TUF or turning into an overloaded feature collection.

Appendix A. A compact example of the full PrepStack flow

Example target: Backend Developer Intern

1. User input: the learner selects Backend Developer Intern and pastes a job description.

2. Analyzer output: required competencies include REST APIs, authentication, database modelling, indexing, caching, testing, Git and deployment.

3. Gap diagnosis: the learner has evidence for REST APIs and Git, is learning authentication, and has no evidence for indexing or testing.

4. Plan: PrepStack schedules a DBMS note, a short checkpoint, an API testing task and a project milestone based on the learner's available time.

5. Learning: the learner reads the note and completes a checkpoint explaining when an index helps and when it may not.

6. Practice: the learner marks an external SQL exercise attempted/solved and records confidence. PrepStack labels it as self-reported practice, not automatically verified code.

7. Project: the learner adds indexing and query measurement to a backend project, records a repository link and explains the trade-off.

8. Communication: the learner drafts a resume bullet from the actual implementation and adds a story about debugging a slow query.

9. Readiness update: the indexing competency moves from "not assessed" to "demonstrated" only when the selected evidence meets the defined checkpoint/rubric.

This is the intended difference: the platform does not merely say "complete DBMS indexing". It connects the concept to practice, a real project, an explainable status and a defensible interview explanation.

Appendix B. A simple first-pass competency status model

Status	Meaning	Typical next action
Not assessed	No meaningful evidence has been recorded.	Start with a diagnostic question or learning asset.
Learning	The learner has started the content but has not completed an application checkpoint.	Finish the concept and attempt a checkpoint.
Practised	The learner has attempted relevant practice, but evidence is incomplete or self-reported.	Repeat, explain the approach or complete a stronger checkpoint.
Demonstrated	Evidence meets a defined standard for the competency.	Use it in a project or interview explanation; refresh later if needed.
Needs refresh	Previously demonstrated evidence may be old or the learner has struggled on a later check.	Schedule a short revision and re-check.

Appendix C. Competitor research discipline

When you revisit TUF or any competitor, use a repeatable audit sheet. Record the date, plan/account tier, exact workflow, screenshots or notes, and what you personally verified. Separate official marketing claims from observed behaviour. Do not infer that a feature is missing simply because it is not visible on the homepage.

• Onboarding: how does the product learn the user's level, goal and available time?

• Diagnosis: does it map a target role or JD to specific skills and gaps?

- Planning: how are tasks selected, prioritized, estimated and rescheduled?
- Practice: what can the learner do inside the platform and what is external?
- Verification: what evidence distinguishes attempted, completed and mastered?
- Projects: are project milestones connected to role competencies and portfolio artifacts?
- Communication: does the product help the learner explain verified work in resumes and interviews?
- Trust: are recommendations explainable, privacy controls clear and progress data exportable/deletable?

The purpose of competitor research is not to find features to copy. It is to understand which user problems are already well served, which remain frustrating, and where PrepStack can deliver a clearer or more trustworthy outcome.